

Protocol-Governed Computing: An Architecture for Closed-Loop Governed Transformation

© 2026 Bhash Ganti

Contact: bachipeachy@gmail.com

ORCID Profile: <https://orcid.org/0009-0007-3810-6520>

Preface

Software changes continuously, yet software engineering has traditionally treated construction, execution, and evolution as largely separate concerns. This paper presents an architecture for software evolution—how running software changes over time while remaining governed, verifiable, and executable—in an era in which AI and LLM-based coding assistants can automate software implementation at increasing scale. When implementation can be generated, modified, and regenerated automatically, the question of who governs the act of change becomes the architectural question. It is the second of three companion papers: the first addresses deterministic declarative execution [Ganti, 2026l], and the third addresses realization of the architecture as a functioning normative platform [Ganti, 2026m].

Most of what follows can be read without any prior knowledge of Protocol-Governed Computing (PGC). The architecture is general. It concerns the relationship between a business problem and the running software that addresses it, and that relationship is the same whether the implementation is written by humans, generated by AI, or produced through fully automated software construction. A reader interested in software architecture—and skeptical of any particular platform—is exactly the reader this paper is written for.

Still, a few words of orientation will help, because the architecture was worked out and validated in a working reference implementation — open source at <https://github.com/protocol-governed-computing>, and described in the realization companion [Ganti, 2026m]. The architecture is offered here on its own terms, but nothing in it is hypothetical: the transformation stages, the compiler, the validated candidates, and the promotions this paper argues from are all inspectable.

A protocol-governed system separates business intent from behavioral implementation. Business intent is transformed into an executable behavioral model that is described declaratively as executable protocols rather than embedded in imperative application code. A compiler resolves, validates, and compiles those declarations into a single authoritative artifact—the Protocol Snapshot. A runtime then executes that snapshot and nothing else. The runtime carries no behavioral knowledge of its own; every behavior it performs was determined, validated, and compiled before execution began.

In a protocol-governed system, nothing runs that was not first declared, validated, and compiled into the Protocol Snapshot.

This paper takes up the first half of that sentence: how a Protocol Snapshot comes to be declared, validated, and compiled at all. Its companion—*An Architecture for Deterministic Declarative Execution* [Ganti, 2026]—takes up the second half, and treats *nothing runs that was not* as a promise execution itself must keep. Earlier papers in this series established the discipline for construction and execution. They did not fully answer what happens when the Protocol Snapshot itself must change—or who governs the act of changing it. That is the question this paper takes up.

The answer, like its companion's, turns on a single rule about authority:

Only a human-directed author may create meaning; the machine may derive it, never originate it.

That rule is not specific to any implementation. It is an architectural model for governed software evolution—and, increasingly, a necessary foundation for software whose implementation may itself be produced and transformed by machines.

In plain terms

Here is the idea before the argument follows.

Every way we have of changing software runs through the same intermediate object: someone writes down what the system should do — a requirements document, a design, a model, a ticket, a prompt — and someone or something else turns that into code. The written thing and the running thing then have to be kept in agreement, forever, by hand. They never quite are. That gap is the source of most of what we call software maintenance: the spec that no longer matches, the behavior nobody meant, the change that broke something unrelated.

This paper proposes removing that intermediate object entirely, and it comes down to one rule about who is allowed to decide what things *mean*.

A change does not start with someone authoring a specification. It starts with a **business problem** stated in ordinary business language, and with an existing system that already runs — the **baseline**. From there a governed pipeline carries the problem forward in stages, each stage adding exactly one layer of meaning and contradicting none of the earlier ones, until what comes out the far end is not a document but the compiled artifacts of the next running system. Then those artifacts are validated and **promoted**, and the system that results becomes the baseline the next change starts from. The loop closes.

The rule that makes this safe is the **Knowledge Partition**: only a human-directed author may create meaning. Machines — including AI — may derive, transform, check, and compile, but may not decide what something means. Every derived fact has to say where it came from, so the whole chain from business problem to running behavior can be walked backwards.

What follows from that rule without further effort:

- **Nobody separately writes a specification**, so no specification can drift out of agreement with the system.
- **The system cannot change except by promotion**, so there is one moment of change and it is recorded.
- **Every fact in the running system traces back** to a human decision or to a derivation from one.
- **AI is optional and replaceable**. It is one way to supply human-directed judgment — a worker, not a foundation. Swap it out and the guarantees are unchanged, because none of them rest on it.

One clarification, because the claim is easy to misread. The specification does not disappear. Every job the traditional specification did — stating intent, fixing scope, recording decisions, defining behavior precisely enough to build from — still gets done. What changes is *where* it is done and *who authors it*: those jobs are absorbed into the stages of the pipeline as derived, validated, machine-checkable artifacts, each one

traceable to the human decision it came from. The system is therefore fully specified — more completely and more literally than a prose document could specify it, since the specification is what compiles and runs. What is removed is the *separately authored, hand-maintained* specification: the parallel document that had to be written by a person, kept in agreement with the code by a person, and that always eventually was not. Nobody writes that document any more; the pipeline produces its content as a by-product of transformation.

The enduring artifact is the surprising part. It is not the code, and it is not a document. It is the **transformation** itself — the governed process that turns a problem into a baseline. Running systems become successive products of it. Software stops being a thing you maintain and becomes a thing you re-derive.

The rest of the paper makes that precise: why the specification was the load-bearing failure (§3–§4), what replaces it (§5–§6), who may create meaning (§7), why derived facts can be trusted (§8), what performs the derivation (§9), how a candidate becomes the system (§10), and which claims the evidence actually supports (§15).

Abstract

The primary engineering artifact is no longer software. It is the governed transformation that produces software.

Software engineering has always relied on a specification as the authoritative bridge between a business requirement and an executable system. For fifty years, methodologies have tried to shrink the gap between the two — through requirements engineering, computer-aided software engineering, model-driven development, domain-specific languages, low-code platforms, and, most recently, AI code generation. Each narrowed the gap. None closed it, because each still produces a *separately authored* specification that must be kept synchronized with an implementation.

This paper presents an architecture that removes the separately authored specification altogether. It treats software evolution not as specification authoring but as the *governed transformation* of an existing executable baseline into the next one. Beginning from a business problem, the transformation enriches meaning in governed stages until it yields executable protocol artifacts. Those artifacts are admitted, validated, promoted, and become the baseline for the transformation that follows, so the system evolves in a closed loop rather than through open-loop change. The running system is a *product* of the transformation; the transformation — not any document, and not the code — is the thing that endures.

The architecture rests on a single organizing rule about *who is permitted to create meaning*, from which its main properties follow. The paper develops that rule and its consequences, positions the result against fifty years of prior attempts, and argues that governed transformation is a practical realization of specification-driven development in which governance replaces the manually authored specification. The model depends on no particular technology and on no artificial intelligence: AI, where it appears, is one interchangeable way to supply human judgment, never the source of the architecture's guarantees. The model has been exercised on multiple completed protocol transformations; a worked example is given in an appendix so that the architecture in the body can be evaluated on its own terms.

1. Positioning — an architecture, not a platform

This is the sixth conceptual paper in the series. The five before it introduced the components: constitutional governance and the layered stack; the compiler that turns declarations into an admissible boundary; the runtime that executes that boundary without domain knowledge; the inversion that puts protocol before implementation; and the governed pipeline by which a protocol evolves.

Those papers describe parts of a system. This one describes something more general than a system.

The contribution here is an *architecture for software evolution*. Protocol-Governed Systems is its reference implementation — the setting in which it was discovered, built, and stressed — but the architecture is separable from it. A reader may accept the architecture and reject the implementation, or the reverse. That separation is deliberate. Implementations age; architectures, if they are any good, do not. The claim this paper defends is meant to outlive the particular tools that first demonstrated it.

The order of exposition follows from this and is worth stating once, because it governs the whole paper: the architectural claim comes first, and the reference implementation appears only as evidence that demonstrates it — never the reverse. Each section states what the architecture requires before it shows how the implementation satisfies the requirement, and the mechanics of that implementation — its repositories, commands, and artifact formats — are confined to the appendices by design. A reader who never intends to run it should still be able to hold the architecture whole, and to build a conforming implementation on entirely different technology from §16 alone.

Stated plainly, the claim is this:

A governed transformation can replace the traditional software specification as the enduring engineering artifact, while executable baselines become successive products of that transformation.

The rest of the paper builds that claim slowly, one question at a time.

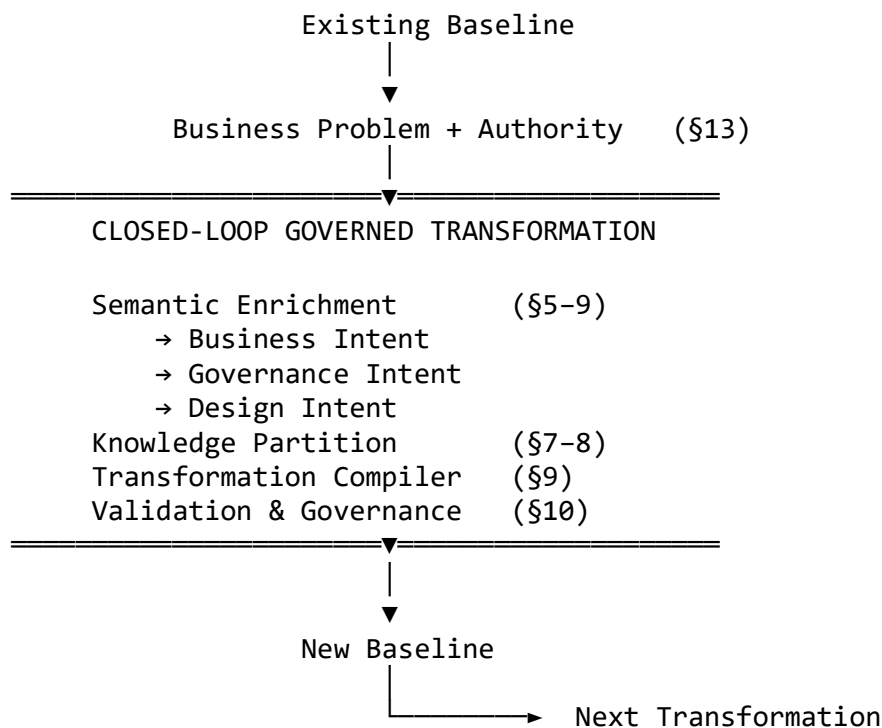
Five named ideas carry the argument, and it helps to have them in view from the start:

- the **Knowledge Partition** (§7) — only a human-directed author may create meaning;
- **Derivation Provenance** (§8) — every derived fact declares where it came from;
- **Progressive Semantic Enrichment** (§9) — each stage adds one layer of meaning and contradicts no earlier one;
- **Promotion as Closure** (§10) — the system changes only when a validated candidate is promoted;
- **Authority without Authorship** (§13) — the human owns every decision and authors no specification.

The paper reaches them by following a single chain of questions, each section ending by posing the next section's. If the specification is removed (§3), what fills the gap it left (§5)? Who is allowed to create the meaning a transformation needs (§7)? If only one participant creates it, why trust what it produces (§8)? What performs the derivation (§9)? And how does a derived artifact become the system (§10)? A reader who loses the thread can re-find it here. For the skeptic who wants the full claim-set before the argument, §16 states the architectural properties as requirements any conforming implementation — this one or not — must satisfy; a complete worked example runs from business problem to baseline in Appendix D.

One picture carries the whole thesis, and the rest of the paper is an unpacking of it. Everything above the box is what the human supplies; the box is what the paper explains, element by element; everything below the box is what the transformation produces and re-enters.

Figure 1 – Conceptual Architecture of Closed-Loop Governed Transformation



A reader can hold this one diagram in mind and treat each following section as a magnifying glass on one element of the central box.

2. Context

The prior papers left a question unanswered, and it should be named precisely, because the whole paper is an attempt to answer it.

Once behavior lives in protocol rather than in code, and once the process of changing the protocol is itself governed, a deeper question appears beneath both. *What, exactly, does the human contribute — and what is compiled from it?*

The earlier change-management work gave a partial answer. It showed that a model of the business could serve as the single canonical artifact of a change, and that the governance documents downstream of it could be projections of that model rather than independently written specifications. That closed the loop at the level of process.

But it described the human's contribution loosely, as "business intent," as though intent were the input. Later work showed that intent is not the input. Intent is *derived*. The human supplies something earlier and simpler — a problem, and the authority to resolve it — and the transformation derives the intent.

This sharpens the earlier position rather than contradicting it. The prior paper held that progressive enrichment is irreducibly human-driven — that no machine can infer what business rules a domain must enforce. That remains true here; the Knowledge Partition (§7) is its precise statement. What changes is where the human's contribution is *located*. The human's judgment still enters at every stage; the human's *input* is only the problem and the authority. Judgment throughout; input only at the start.

That shift, from intent-as-input to problem-as-input, is small to state and large in consequence. It is the difference between a system that helps you write a specification and a system that makes the specification unnecessary. This paper follows the consequence to its end.

3. The intermediate artifact nobody removed

Consider what every software methodology has in common.

Waterfall gathers requirements, writes a specification, produces a design, and builds. Agile does the same work in smaller loops and calls the pieces by friendlier names. Model-driven approaches make parts of the specification executable. The differences among these methods are real and much discussed. But underneath the differences sits a shared assumption so constant that it is rarely noticed at all.

In every one of them, a human writes down what the software should do, as an artifact distinct from the software itself.

Requirements → Specification → Design
→ Implementation → Testing → Deployment

That artifact — the specification — is the intermediate that no methodology has removed. Each new method has tried to make it cheaper to write, easier to trace, or partly executable. None has eliminated it. And because it is a separate description of the same behavior the code also describes, the two must be reconciled continually, forever. This is the synchronization problem, and it has never been solved. It has only been managed.

This paper does not offer a better way to manage it.

This paper removes the artifact that creates the problem.

The claim needs one immediate clarification, because without it a reader will object within a page. Removing the specification does not mean the system has no specifications. It means the system has no *separately authored* one. The distinction is the hinge of the entire argument, and §12 returns to it in full. For now it is enough to say: the governed declarations that the transformation produces are themselves precise specifications — but no human wrote them as a separate step, and there is nothing for them to drift away from, because they *are* the executable truth.

If the specification is removed, though, a question opens immediately. It will take the rest of the paper to answer, and it deserves a pause before going further:

If no one authors the specification, what does the human do, and what fills the gap the specification used to occupy?

(A reader who wants the concrete before the abstract can turn to the worked example in Appendix D, which runs a single real transformation from problem to baseline; the argument below does not depend on reading it first.)

4. Why the specification was the load-bearing failure

Before answering that question, it helps to see why removing the specification is worth the trouble. The specification is not a neutral convenience. It is the structural point where several well-known failures originate.

Traditional development is *open-loop*. Requirements go in at one end; a running system comes out the other. What comes out is never systematically compared, in a governed way, against what went in. The loop is never closed, and four familiar problems live in the gap.

The first is **requirements leakage**. A requirements document has no structural guard against implementation creeping in. “The system shall store records in a relational database” reads like a requirement but is a design decision. Nobody smuggles it in on purpose; there is simply no mechanism that forbids it, so it accumulates.

The second is **rationale decay**. A specification records *what* was decided but rarely *why*. The reasoning — why a boundary was drawn here rather than there — lives in people’s heads and in documents that age. The system inherits the decision and loses its justification.

The third is **governance externalization**. In most organizations, governance is a review wrapped around the engineering. A board approves what engineers already built. The approval records an outcome; it changes nothing about what the system itself knows.

The fourth is **evolution amnesia**. When the time comes to change the system, the thing being changed is opaque. What was it meant to do? What was deliberately left out, and for which later change? Answering requires archaeology, because the answers were never written where the next change could read them.

None of these are failures of skill or care. They are consequences of the specification’s position in the process — a separate artifact, unguarded, unexplained, and external to the system it describes.

Protocol-governed construction already addresses the first three, at the moment of building. But the fourth returns the instant a system must change. And if the change process is itself ungoverned, all four return together. Closing the loop, then, is not a refinement. It is the only way the guarantees survive contact with time.

So the specification has to go. Which brings us back to the question it left open.

5. Evolution is transformation, not authoring

If the human no longer authors a specification, what *is* the act of changing a system?

The answer this paper proposes is a change of category. Evolving a system is not authoring a new description of it. It is transforming the system from one state into the next.

This has a precondition that the architecture supplies and that deserves to be stated outright, because much follows from it: **nothing is greenfield**.

In ordinary software, “greenfield” is a meaningful idea — an empty directory, an open design space, no constraints. In a governed system that situation never occurs, not even for the very first change. There is always a baseline: the compiled, authoritative artifact that defines exactly what the system currently is. Even the first change to an empty domain is a change *to* the constitutional substrate that was already present. One never builds from nothing. One always transforms something.

This raises the obvious question of where the very first baseline comes from — the cold start. The answer is that the substrate is itself the baseline, and it is small. Before any domain exists, there is a `Baseline_v0` consisting only of the constitutional governance artifacts: the federation boundary, the execution topology, the invariants every later artifact must respect. That constitution is authored once, as the founding act of the platform, and from then on it is transformed like everything else. The first domain transformation is not a greenfield build; it is a governed transformation of `Baseline_v0`, adding the first domain under rules that already exist. Bootstrapping the constitution is therefore the only genuinely foundational authoring the model requires — and it is a one-time act, outside the loop the rest of the paper describes, precisely so that every transformation *inside* the loop has a baseline to transform.

From this a single, near-axiomatic result follows. It is the one principle the paper asks the reader to hold onto, because the remaining sections are consequences of it. It is stated as a principle, not a theorem, deliberately: it has no formal apparatus behind it, and the prior work states the same content as a definition — if the Protocol Snapshot does not change, the system is invariant by definition. Naming it honestly avoids claiming a derivation that does not exist.

Baseline Transformation Principle. Every executable state of a protocol-governed system is reachable only through a governed transformation of an existing executable baseline.

The principle has a plain corollary that is more useful than it first appears. If the baseline does not change, the system has not changed. A modification to documentation, tooling, or surrounding infrastructure that produces no new compiled baseline has changed things *around* the system, not the system itself. The unit of engineering, therefore, is not a document. It is a transformation of an executable baseline into another executable baseline.

That last phrase — *from one executable baseline to another* — is what eventually lets the loop close, because the output of a transformation is the same kind of thing as its input. But the loop comes later. First, a smaller matter of vocabulary.

6. Why “transformation” and not “change”

Be deliberate about the word, because “change” quietly carries assumptions that this architecture denies, and keeping the wrong word invites the wrong intuitions.

“Change” suggests *discretion* — that an engineer decides, locally, what to alter. Here the alteration is derived under governance, not chosen. “Change” suggests *locality* — that a modification can be confined to the file one happens to be editing. Here the reach of a transformation is bounded by the structure of the protocol, not by anyone’s mental map of the code. And “change” suggests *freedom* — the greenfield openness to build however one likes. The Baseline Transformation Principle has already ruled that out.

A transformation is a more disciplined thing. It is defined by what it holds constant and what it adds:

- it **preserves identity** — the domain and its versioned artifacts remain themselves;
- it **preserves invariants** — no constitutional rule is broken in the passage from one baseline to the next;
- it **introduces governed capability** — whatever new behavior appears is admissible, validated, and attested before it exists.

This is a richer notion than change management, and the richness is what makes the loop possible. Because a transformation turns a baseline into another baseline of the same kind, the result of one transformation is a suitable input to the next. “Closed loop” stops being a metaphor and becomes a structural fact.

We now have the shape of the thing: a business problem enters, a transformation runs, a new baseline results. But the interesting questions are inside the transformation. The first and most fundamental one is about meaning.

7. Who is allowed to create meaning?

Here is the question on which everything else turns.

A transformation takes an informal business problem and produces precise, executable artifacts. Somewhere along the way, meaning is created — a judgment is made that *this* existing capability is relevant to the problem, that *that* boundary belongs here, that a new behavior is what the business actually wants. These are acts of interpretation. They cannot be read off from the existing structure of the system; they are decisions about what the structure should become.

So the question is simple and consequential: *who, or what, is permitted to make those interpretive decisions?*

The architecture gives an uncompromising answer, and this section makes only this one point, because the point is strong enough to stand alone:

Only one participant in the entire architecture may create meaning: the human-directed author of the transformation. Nothing else may.

Three roles recur through the rest of the paper, and it helps to fix them here. The **platform** is the producer of deterministic knowledge — everything computable from what already exists. The **worker** is the interchangeable supplier of judgment: a human engineer, an AI agent, or the two together (§11 returns to this). The **author** is the worker acting under human authority in a particular transformation — the participant the Knowledge Partition names as the sole producer of meaning. Where the paper says “author” it means judgment exercised under authority; where it says “worker” it means the same function considered apart from who fills it.

Call this the **Knowledge Partition**. It draws a hard line between two kinds of knowledge. On one side is *deterministic knowledge* — facts that can be computed from what already exists. What artifacts are present. How they depend on one another. Which declared concepts resolve to nothing. On the other side is *judgment* — the assignment of relevance and intent, which cannot be computed from structure at all.

The partition assigns each kind to exactly one place. Deterministic knowledge is produced by the platform, mechanically and reproducibly. Judgment is produced by the transformation author, and only there.

The reason the platform is *forbidden* to create meaning — not merely discouraged — is that meaning genuinely cannot be derived from structure. One can compute structure from structure all day. One cannot compute *relevance* from it, because relevance depends on the intent of the change, and the intent is not in the graph. A system that tries to guess relevance from structure will confidently surface the wrong things. The only honest move is for the platform to present everything it can determine, and to leave every judgment of significance to the author.

That single rule — one producer of meaning, and it is never the machine — is the foundation the rest of the architecture is built on. Everything that follows is a consequence of taking it seriously. And taking it seriously raises the next question immediately.

8. If meaning has one producer, why trust what it produces?

Concentrating all judgment in a single author is powerful, but it looks, at first, like a risk. If one participant creates all the meaning, and the meaning is not computable, how does anyone downstream *trust* the derived facts? What stops the author's interpretations from becoming an unaccountable black box — exactly the opacity we were trying to escape?

The answer is a discipline on how meaning may be recorded, and it belongs as a rule of its own:

Every derived fact must declare where it came from. No fact may be merely asserted.

Call this **derivation provenance**. Whenever the transformation establishes a property, that property must carry its source — either a structural fact it was computed from, or a governed decision an author made. Not “this belongs to that subdomain,” but “this belongs to that subdomain *because* its declared ownership says so.” Not “this capability is required,” but “this capability is required *because* the author, holding authority, decided it in resolving this problem.”

Two things follow, and both matter.

First, judgment stops being opaque. Because each act of meaning-making leaves a trace back to its origin, one can *audit the reasoning*, not merely the result. The author's interpretations are not a black box; they are a chain of stated derivations, each answerable to a source.

Second, the whole structure becomes composable. Because every fact carries its provenance, each stage of the transformation can build on the stage below it *without re-deriving anything*. It simply reads what the prior stage recorded, provenance and all. Two parts of the system can never quietly disagree about a fact, because neither of them independently invented it. The single-producer rule, which looked like a risk, becomes a guarantee: there is exactly one place meaning is made, and it always shows its work.

We now have a producer of meaning, and a reason to trust what it produces. That raises a mechanical question. If meaning is created in governed, provenance-bearing steps — what actually performs those steps, and turns a business problem into executable artifacts?

9. What performs the derivation: a compiler that enriches meaning

The thing that performs the derivation behaves like a compiler. Not by analogy alone — it is worth saying carefully how it is like a compiler and, more importantly, how it is not.

A traditional compiler takes a program in a high-level language and lowers it, step by step, toward machine code. At each step it *preserves* meaning while changing representation. Source becomes syntax tree, becomes intermediate form, becomes optimized instructions. The meaning is constant throughout; only the form descends toward the machine.

The transformation runs in the opposite direction along the axis of meaning. It starts with something that has very little precise meaning — an informal business problem — and *adds* meaning at every stage, until the result is precise enough to execute.

Traditional compiler
precise meaning, lowered in form
(source → machine code)

Governed transformation
growing meaning, raised toward execution
(problem → protocol)

This is why it is fair to call it a compiler, and why it is a different kind of compiler. It does not translate a language. It *enriches* a problem. Each stage of the transformation adds exactly one layer of meaning that was not there before, and — this is the crucial constraint — contradicts nothing that earlier stages established.

The stages, at the altitude of the architecture rather than any implementation, run roughly like this:

Business Problem
→ Business Understanding (what exists, what the problem touches)
→ Business Intent (what behavior may occur, under what rules)
→ Governance Intent (which part of the system owns what)
→ Design Intent (the concrete shape of the artifacts)
→ Construction (build-ready form)
→ Protocol → Snapshot → Execution

For readers of the prior work, these altitudes map onto the governed pipeline directly. *Business Understanding* corresponds to the discovery stages — Domain Model, Analysis, Business Model. *Business Intent*, *Governance Intent*, and *Design Intent* keep their names. *Construction* is the Authoring Mandate and the authoring phase it drives. The names differ only where a general reader is better served by a plainer word. This enrichment, from problem through Design Intent, produces what the prior work named the Business Model as the single canonical artifact of a change; here that model is treated not as one named stage but as the continuous derivation the whole ladder performs.

Two rules keep the enrichment honest, and together they are what make the process a compiler rather than a series of loosely related documents. **Every stage adds information.** And **no stage contradicts an earlier one.** A design decision may *answer* a question the business model left open; it may never *override* a business rule. When a later stage cannot proceed without contradicting an earlier one, that is not a problem to be smoothed over. It is a signal that the earlier stage was incomplete, and the transformation goes back to it.

The discipline that enforces this has a useful image, and a name carried from the prior work: the *purity ladder* of vocabulary. At the bottom rung, only business language is allowed. Each rung up admits exactly one more kind of vocabulary — provisional capability names, then ownership and placement, then binding identifiers, then build order — and never more than one. A stage that reaches for vocabulary above its rung is out of bounds, and the violation is mechanical to detect. The one standing exception, at every rung, is that anything which *already exists* may be named exactly, because naming what exists is observation, not design.

Stated as a table, the ladder is short enough to hold in view:

Rung	Newly admitted vocabulary	Still out of bounds
Business problem	business language only — what is needed and why	any capability, owner, identifier, or ordering
Business understanding	what already exists in the baseline, named exactly	anything proposed but not yet existing
Business intent	provisional capability names — <i>what the thing will be</i>	which part of the system owns it
Governance intent	ownership and placement — <i>whose it is, where it lives</i>	binding identifiers and wiring
Design intent	binding identifiers, artifact shapes, wiring	build order and construction detail
Construction	build order — <i>what to build, in what sequence</i>	—

Each row admits exactly one more kind of vocabulary than the row above it, and nothing skips a row. A stage that uses a term from a lower row has reached above its rung, and that is a mechanical violation rather than a matter of taste.

This single enriching process is the *transformation compiler* named in Figure 1 — the core engine of the platform, whose job is to execute the progressive semantic enrichment stage by stage. Underneath it sit two concrete compilers, and they should be named rather than folded into the metaphor. The first, the **semantic compiler**, turns the enriched model into *candidate* artifacts — proposed pieces of protocol that do not yet officially exist. The second, the **admission compiler**, tries to *admit* them: it assembles a workspace, overlays

the candidates on the real system, and asks whether the system can consume what was proposed. The admission compiler is not a new engine invented for change. It is the same constitutional compiler that governs everything else — the compiler of the earlier papers — now operating over Baseline U Candidate Delta. That it cannot tell which pieces are new and which were already there is the whole point; it just compiles the whole. Because of that, *admissibility* has exactly one arbiter — admission — and no candidate is admissible until it passes.

Between the enrichment and the candidate artifacts there is a boundary worth naming, because the two sides fail differently and the difference decides who fixes what. On one side, the derivation can fail by being *incomplete or contradictory* — a register that does not say what the stage owed. On the other, it can fail in a way that reads as success: every register well-formed, every rule satisfied, and the design still does not fix the artifact it describes. The first is a fault in what was written. The second is a fault in what the language *allowed* to be left unwritten, and no amount of re-authoring a register will find it.

That second failure is only visible if the boundary is measured rather than assumed. **The proportion of the facts an artifact requires that the design actually states is a quantity**, and construction must refuse to proceed below it. The reason is the paper's own thesis turned on itself: a generator that supplies a fact the design omitted has begun to author design. It becomes a second producer of meaning, unreviewed and ungoverned, sitting *below* the protocol where §4 argued the drift was finally removed. Specification drift does not need a specification document to reappear; it only needs somewhere unattended for a decision to be made. Measuring determination is what keeps that place from existing.

One boundary must be drawn precisely here, because the paper later depends on it. Admission settles **buildability**: whether the proposed protocol is structurally and constitutionally sound. It does not, and cannot, settle whether the built system does what the business problem asked. That is a different question — behavioral, not structural — and §10 gives it a stage of its own. Admission is the sole arbiter of admissibility, not the sole arbiter of every kind of correctness; the paper is careful about the difference from here on.

So the transformation has a producer of meaning, a reason to trust it, and a mechanism that enriches a problem into admissible artifacts. One question remains before the loop can close. Admissible artifacts are not yet the system. How do they *become* the system, and how does the next transformation begin?

10. Closing the loop: promotion as the moment of change

Admission proves a candidate is buildable. It does not prove the candidate does what the business problem asked, and it does not make it the system. Two things stand between an admitted candidate and a changed system, and both matter.

The first is **implementation**. A capability the transformation declares is a governed contract — its inputs, outputs, and outcomes are fixed — and something must supply the conventional code that satisfies it. This is where implementation authoring sits in the loop: after admission, before the candidate can be trusted as the system. Its status must be stated exactly, because a reader will otherwise suspect the specification has merely moved downstream. Implementation is *realization* of an already-governed contract, not a source of behavior; the behavior was fixed upstream, in the protocol. And the compiler enforces conformance of implementation to contract, so the drift the architecture removed above the protocol cannot reappear below it. The contract between protocol and code is not a separately authored specification kept in agreement by diligence — its agreement is compiler-maintained. There is no second description to reconcile here either.

A consequence of this deserves stating, because it is invisible in a system's first change and unavoidable in every later one. A transformation schedules what it must build — and an artifact that already exists cannot be scheduled for authoring, because authoring it again would create a second artifact under the same name. Amendment and scheduling are therefore different acts, and a loop that realizes only what it schedules will leave every amendment silently unapplied while reporting that it built everything it was asked to. After the first change, most of what a transformation does is amend.

This forces a discipline that is easy to get backwards. An amendment must be a **whole redeclaration** of the artifact as it will be, not a patch against the artifact as it is. The temptation is the reverse — state only what changes and let the builder merge it into what is already there — and it must be refused, because it would make the built artifact a function of the design *and the current system* rather than of the design alone. The same transformation would then build differently against two systems, and determination would stop meaning anything. The price of the discipline is that a design must restate what it is not changing; the protection against paying that price carelessly is to compare a redeclaration against what it replaces and refuse one that quietly drops what it did not mention.

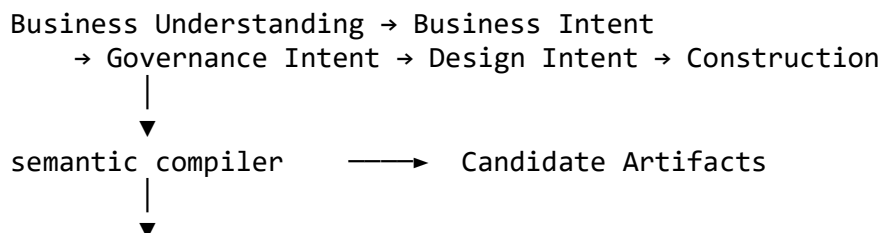
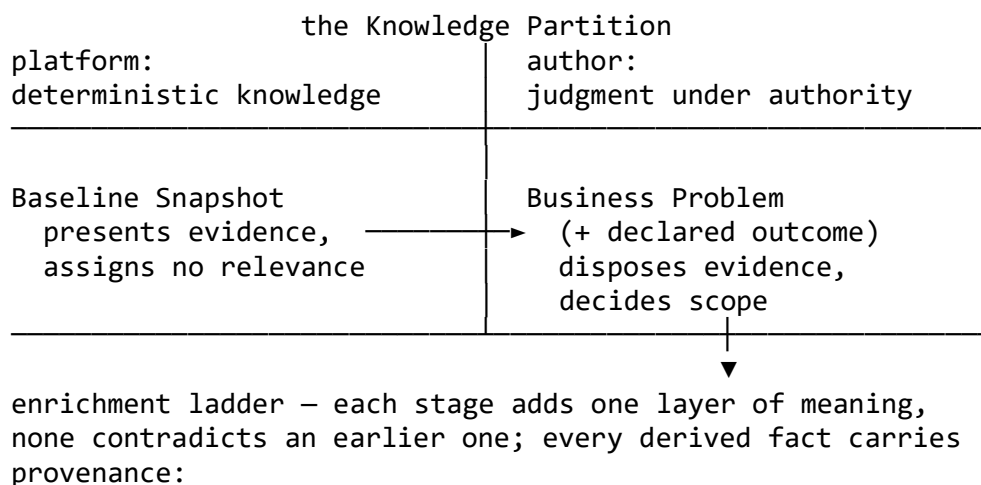
The second is **validation against the problem that initiated the change**. This is the stage the loop names Validation, and it is not a second admission. Admission asks a structural question: is this buildable? Validation asks a behavioral one: does the built system produce the outcome the business problem declared? For that to be checkable, the problem must carry, from the outset, a declared **outcome** — the acceptance boundary against which closure is later judged. Execution validation runs the admitted, realized candidate and observes whether its behavior satisfies that boundary. Only then is the candidate a genuine answer to the problem rather than a merely well-formed one.

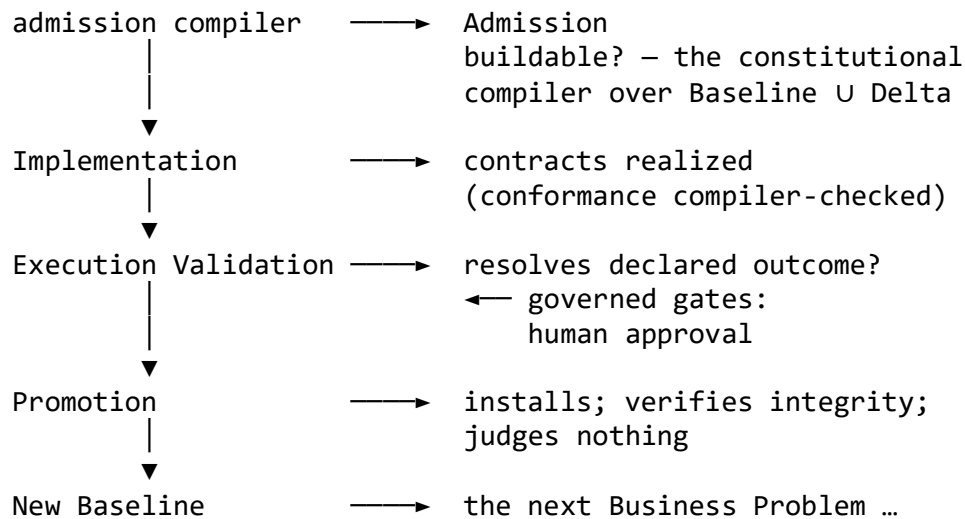
It is worth being exact about *how* the outcome is represented, because this is what makes the loop close tightly rather than by inspection. The outcome is not an informal note held to one side; it is itself a governed part of the transformation, declared with the problem and carried with provenance like every other derived fact. And the check against it is not a separately written test suite. The same transformation that produces the artifacts also produces the conformance under which they are judged — the compiler generates conformance from the governed declarations, and execution validation observes the realized candidate against that generated conformance and the declared outcome together. The test of correctness is derived from the same process as the thing being tested, so there is no hand-authored oracle to drift from the system it checks, any more than there is a hand-authored specification above it.

This is the sense of “closed loop” that §4 demanded, and it is worth being exact that it is a *different* closure than mere type-matching. Open-loop development never compares what comes out against what went in. Here that comparison is a governed stage: the candidate’s observed behavior is checked against the outcome the problem declared, and a candidate that builds cleanly but does not resolve the problem does not pass. *Type* closure — output is the same kind of artifact as input — lets the cycles compose. *Semantic* closure — the output answers its problem — is what makes each cycle worth composing.

The full loop, at last, looks like this (Figure 2) — the same architecture as Figure 1, now with its internal stages and both arbiters made explicit:

Figure 2 – The closed loop of governed transformation, in detail.





Two authorities govern this passage, and it matters that they are two and not one. Admission is the sole arbiter of **admissibility** — structural and constitutional soundness. Execution validation is the sole arbiter of **behavioral adequacy** — that the system resolves its problem. Promotion is neither, and the two do not overlap. The paper’s earlier shorthand, “one arbiter of correctness,” was too strong: there is one arbiter of buildability and one of behavior, each final within its own question.

Human authority enters this passage at declared points, not diffusely. The transformation places governed **approval gates** where the derivation is reviewed as a whole and locked — the point at which the design is accepted, and the point at which the mandate to build is granted. These gates are the operational form of the human’s authority: authority is exercised not by editing artifacts but by approving, at named points, a derivation the human did not author. This is what “authority without authorship” (§13) looks like in mechanism.

Promotion is the last step, and the smallest. It installs the already-admitted, already-validated candidate as the new baseline. It re-examines nothing and re-judges nothing; every question of correctness was settled upstream — structural at admission, behavioral at validation. Promotion verifies integrity — that what is installed is exactly what was validated — and does no more. Where an implementation realizes promotion by rebuilding the workspace snapshot, that rebuild is a mechanical re-homing that introduces no new judgment; it is not a third occasion on which correctness is decided. The toolchain comparison is close but should be stated precisely: a software release ships the executable that was already validated and verifies its integrity — checksums, signatures — without re-judging it. Promotion is that, and only that.

Promotion is easy to overlook, and overlooking it misses where the loop actually closes. A candidate can be fully derived, admitted, and validated and still leave the system exactly as it was — because, by the Baseline Transformation Principle, only a change to the baseline is a change to the system. Validation says “this *does* resolve the problem.” Promotion says “this *is* the system now.” Until promotion, the old baseline stands.

Now the loop is genuinely closed, and closed in a way traditional change management is not. Its output is the same kind of artifact as its input, so the end of one transformation is the beginning of the next. Four properties distinguish it. It is **authority-bearing**: every stage leaves a governed record that can be cited and checked. It is **scope-contained**: the transformation names exactly what must be built, and scope cannot silently grow. It is **evidence-complete**: the record of a transformation is a full, auditable chain, not a trail of commit messages. And it is **self-referential**: the transformation process is itself governed by the same machinery it applies to everything else — the process that changes the system is governed exactly as the system is.

With the loop closed, step back from the machinery to see what has actually been achieved, because it is something the field has been reaching for since it began.

11. AI is a worker, not a foundation

One clarification belongs here, before the larger claim, because readers will wonder where artificial intelligence fits.

Nothing in the preceding sections required AI. The transformation needs a source of judgment — someone to create meaning under the Knowledge Partition — but the architecture does not care what that source is. It may be a human engineer. It may be an AI agent. It may be a person and a model working together. The interface a worker presents is the same in every case: it turns a governed stage into a governed output, and the output carries the same authority no matter who or what produced it.

This must be stated precisely, because it is easy to mistake the architecture for an AI system. It is not. AI is one convenient way to supply worker judgment, and a rapidly improving one, but the guarantees in this paper come from the Knowledge Partition and the governed transformation, and those hold whether the worker is a person or a model. If every AI system in existence vanished tomorrow, the architecture would stand, and a room of human authors could run it.

It is worth saying *how* that independence is achieved, because asserted independence is cheap and mechanical independence is not. It rests on three separations, none of which involve the worker at all. The rules a stage must satisfy are **declared as data** — each naming itself, what it governs, and the mechanism that evaluates it — so admissibility is decided by a rule set rather than by whoever is drafting. The set of evaluating mechanisms is **closed**, so an unrecognised one fails rather than being quietly skipped; a skipped rule is indistinguishable from a satisfied one. And where a rule set is read in more than one place, the copies **derive from a single declaration** and any divergence is detectable, because two readers disagreeing about whether a document is admissible is worse than either being wrong.

Given those, what a worker may and may not do becomes exact. It may draft the prose that fills a register. It may not decide whether the result is admissible, and it may not answer a question the business has not answered — an unanswered question is recorded as a question, never filled in and never hedged, so it cannot be quietly resolved by whoever happens to be at the keyboard. The determinism that follows is precise rather than total: given the same human answers against the same baseline, any worker yields the same governed artifacts. Two workers will word a rationale differently. Nothing governed will differ.

The protocol is the system of record. Automation is layered on top of it, never underneath.

12. What this achieves: specification-driven development, finally

Step back and look at the whole shape.

For half a century, the field has tried to close the gap between what a business wants and what software does. Each generation of tools attacked the same gap from a slightly different angle.

Requirements Engineering → CASE → MDD → DSL
→ Low-Code → AI Code Generation → ?

Requirements engineering formalized the writing of requirements. Computer-aided software engineering generated code from diagrams. Model-driven development made the models more executable. Domain-specific languages shrank the gap by shrinking the domain. Low-code replaced written specifications with drawn ones. AI code generation replaced the writing effort with a prompt. Every one of these was a real advance, and every one of them left the same thing standing.

Seen as a whole, this is a single fifty-year research program, and it has had a single hidden premise. Every decade moved the specification *closer to execution* — from prose, to diagrams, to models, to generated code — and each move was treated as progress. But no decade questioned the premise underneath the program: that a separately authored specification should remain the enduring engineering artifact at all. The whole lineage optimized the position of the specification; none of it asked whether the specification should be there. That unasked question is this paper's contribution, and answering it is what finishes the program rather than merely extending it.

In every approach before this one, a human still authors a separate specification.
That is the artifact this architecture removes.

That is the whole difference, and it is easiest to see side by side.

Approach	What the human authors	Agreement with the system maintained by...
Requirements Engineering	Requirements documents	human diligence
CASE	Models plus generated code	human diligence
Model-Driven Development	Platform-independent models	human diligence
Domain-Specific Languages	Programs in a small language	human diligence
Low-Code / No-Code	Visual models	human diligence
AI Code Generation	Prompts, and specifications the code is checked against	human diligence

Approach	What the human authors	Agreement with the system maintained by...
Governed Transformation	A business problem, and authority	construction

The claim has to be made carefully, because stated carelessly it is false and a reviewer will say so at once. This architecture is not free of specifications. It is dense with them — the protocol artifacts are specifications, and rigorous ones. What it removes is the *separately authored* specification: the human-written description that sits beside the system, drifts from it, and demands endless reconciliation. Here, the governed declarations are derived straight from the problem, and they *are* the running truth. There is no second description to keep in step, because there is no second description at all.

The table’s final column, not its middle one, is where the real difference lives, and the distinction is the same criterion §13 makes precise: a description is *separately authored* when its agreement with the running system is maintained by human diligence, and *derived* when its agreement is maintained by construction. Apply it to the two rows a critic will press hardest. A domain-specific language looks like a counterexample — a well-designed DSL program *is* executable truth, with no second description to drift from. But the DSL program is itself a separately authored formal description, and it sits beneath informal requirements that go on drifting above it; the gap has moved, not closed, and its agreement with the business need is still kept by diligence. Model-driven development with full generation is the same case: the model is authored, and its fidelity to the requirement is a human responsibility. Formal specification is the strongest form of “the specification is the primary artifact,” and the most instructive case of all. A TLA+ specification is precise, checkable, and admirable — but it is checked *against* an implementation, not compiled *into* one; the agreement between spec and system is maintained by refinement effort. It is, in this paper’s exact sense, separately authored. Governed transformation is the first arrangement in the lineage where the description’s agreement with the system is maintained by construction rather than by anyone’s diligence.

Two engagements close the lineage. Brooks’s *No Silver Bullet* drew the distinction this whole architecture runs on, under other names: *essential* complexity — the hard conceptual work of deciding what the software must do — versus *accidental* complexity, the incidental labor of expressing it. The Knowledge Partition is that distinction made structural. Judgment, the essence, is conserved and kept with the human; the separately authored specification is accidental, and it is the accident this architecture removes. The paper therefore claims no silver bullet, and the reason is exactly Brooks’s: it removes an accident while conserving the essence, and the essence is where the difficulty always lived. The other engagement is with the current wave of spec-driven AI development — executable specifications, behavior-driven workflows, and the newer spec-first coding assistants. They belong on the same lineage line, at its present end, and they are real progress. But they still produce a human-authored specification beside generated code,

still maintain its agreement by diligence, and still have no governed admission arbiter and no closed loop. They shrink the writing effort; they do not remove the second description.

This is also why the architecture is not merely novel but timely. For most of the fifty-year lineage the second description was expensive to produce, and its expense held the drift in check: a specification and its code moved slowly, so diligence could keep pace. AI code generation removes exactly that brake. When descriptions and implementations can both be produced in seconds, the diligence that held them in agreement is the one thing that does not speed up, and the gap between intent and running system widens faster than any human process can reconcile. An architecture whose agreement is maintained by construction rather than diligence is therefore not an elegance to be adopted at leisure; it is the control plane an AI-accelerated SDLC needs in order to remain governable at all. The role of AI here is unchanged — it is a worker (§11), not the foundation — but its arrival is what makes removing the second description urgent rather than merely desirable.

This is what specification-driven development was always trying to be. The correct name for the shift is not “specification-free.” It is a move from **specification-centric** engineering, where an authored document is the pivot of the process, to **transformation-centric** engineering, where the governed transformation is.

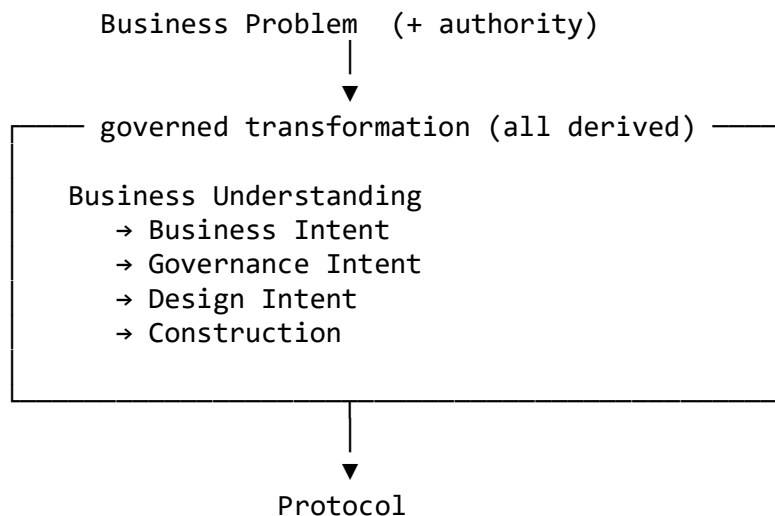
13. What the human actually supplies

If the human authors no specification, it is fair to ask what is left for the human to do. The answer is not “less.” It is the part that was always the real work, with the clerical part removed.

The human supplies two things, and nothing else.

The human supplies a **business problem** — what is needed, why, what already exists that must be respected, and what is deliberately out of scope. And the human supplies **authority** — the standing to make the decisions the problem raises. Which capabilities are in scope. Where a boundary belongs. Whether a new part of the system should exist at all.

What the human does *not* supply is equally important. Not the design. Not the architecture. Not the topology. Not the code. And — the strongest version of the claim — not the business intent either, because intent is *derived* from the problem, not stated alongside it. The transformation is the whole passage below, not a stage inside it:



Here the paper’s central distinction earns a precise criterion, because without one “derived, not authored” sounds like a euphemism for “authored, with extra steps.” The difference is not that the human contributes less meaning — the human contributes all of it. The difference is *how the contribution is held in agreement with the running system*. A specification is **separately authored** when its agreement with the system is maintained by human diligence: someone must keep re-reading it, re-checking it, reconciling it as the system moves. A specification is **derived** when its agreement is maintained by construction: the artifacts are produced from the human’s governed decisions and cannot drift from them, because the decision, its provenance, and the artifact are one chain. Here the human’s every decision is captured in a governed register, carried with provenance, and mechanically projected into the artifacts. What is removed is not the human’s meaning. It is the second copy of it that used to need minding.

This is a sharper division of labor than software has usually managed, and its name is **authority without authorship**. The human owns every decision that requires authority, and authors none of the specification that expresses those decisions. Authority cannot be computed — a machine may propose a plausible purpose, but a proposal is not a decision, and only a human holds the standing to decide. The exercise of that authority is not diffuse: it happens at the governed approval gates of §10, where the derivation is reviewed as a whole and locked. Everything downstream of the decision can then be derived under governance.

The shift is easiest to feel by contrast with management-by-review, the model this replaces. In the traditional arrangement, authority is exercised *after the fact*, by approving artifacts a human has already authored — the board signs off on the design someone wrote, the reviewer approves the document someone drafted. Authority there is a judgment about a finished description. Here authority is exercised *within* the transformation, as a small set of binding decisions that constrain what the derivation may produce — is this capability in scope, does this boundary belong here, should this part of the system exist at all. The human is no longer reviewing documents; the human is making the decisions from which the documents are derived. That is the whole of the shift: from approving authored artifacts to deciding the constraints under which artifacts are authored for you.

The specification is gone. The human is not. What remains for the human is judgment and authority — which is what we wanted people doing all along.

14. The system gets smarter as it is used

There is one more property to draw out, because it is a consequence of the architecture that open-loop development can never have.

Every transformation leaves the system better equipped for the next one. It leaves a richer baseline, a fuller governance record, and — less obviously — a platform that knows more than it did before.

The nearest thing in traditional practice is documentation, and it decays. Governed transformation does the reverse, for a structural reason. Because each transformation records its reasoning with provenance, the next transformation inherits that reasoning as *usable structure*, not as prose to be re-read. Rules discovered while solving one problem become constraints that shape every later one. The cost of understanding the baseline falls with each cycle instead of rising. (Readers of the series will recognize this accumulation as the mechanism behind what earlier papers called the Governance Dividend; the term itself is set aside here — see Appendix A — only because it has been defined differently across those papers, not because the phenomenon is any different.)

There is a second, deeper form of this. Over successive transformations, work that once required a human can migrate, one responsibility at a time, from the worker into the platform — but only work that is genuinely deterministic. The gathering of evidence, the checking of consistency, the diagnosis of a failure: each of these can move from judgment into mechanism, once, and stay there. What is left for the worker is only what truly needs judgment. The Knowledge Partition governs this migration and keeps it honest — nothing that requires meaning is ever allowed to move to the machine, no matter how convenient it would be.

This yields a measurement rare in software architecture — one the architecture makes possible, whether or not any implementation yet reports it as a number. Because every derived property declares its source, a system built this way *can* report how much of its own model it knows deterministically versus how much still rests on judgment, and therefore where the next investment should go. The capability is a consequence of provenance rather than an additional mechanism: the reference implementation already carries the provenance such a measurement would read, and turning it into a reported figure is a projection over data the architecture keeps regardless. Most systems discover what they are missing by failing. This one is structured to discover it by measuring.

15. Evidence

The architecture in this paper is not only argued; it has been exercised.

It was worked out and stress-tested in the reference implementation across a family of completed protocol transformations in a single reference domain — seven in all, six of them carried through to a promoted baseline and the seventh through construction and admission. Those transformations are where the ideas in this paper came from: the Knowledge Partition, the derivation discipline, the two-compiler structure, and the promotion-as-closure model were each discovered at the point where a transformation tried to proceed without them and could not.

Because the argument is meant to stand on its own, the empirical detail is kept out of the main text and gathered in two appendices. Appendix B lists the completed transformations and the reference implementation notes. Appendix D walks a single transformation from business problem to promoted baseline in full, as a concrete illustration of every stage the body describes.

Two observations are worth stating here, because they bear on the architecture’s central claims — stated at the strength the evidence supports, and no further. First, in one transformation a structural pre-filter was used to guess which parts of the system were relevant to the change — a judgment the Knowledge Partition reserves for the worker. It chose against the worker’s own later judgment, surfacing artifacts the author discarded and omitting ones the author kept. This does not *prove* that relevance cannot be computed from structure; a single failed heuristic cannot establish an impossibility. The load-bearing argument for that is the in-principle one in §7 — the intent is not in the graph — and this experiment is best read as an illustration consistent with it, not a confirmation of it. Second, the evidentiary depth of a transformation was raised toward parity with a hand-authored reference not by changing the worker but by moving deterministic work out of the worker and into the platform: the same worker, under the same instructions and the same validation oracle, produced a better-grounded result once the platform carried more of the deterministic load. That is the learning architecture of §14, observed in a single case rather than asserted.

These observations should be read with their limits in plain view. The architecture has been exercised in a single reference domain, by a single practitioner who is also its designer — so the “ground truth” of relevance in the pre-filter observation was that same person’s judgment. There is no comparative baseline against a traditional process, and no external replication. None of this weakens the architectural argument, which rests on its structure rather than on these observations; but it does bound what the observations themselves can show. They are evidence that the model runs and behaves as designed on real transformations — an existence proof and a set of illustrations, not a controlled evaluation. (The transformations summarized here reflect the reference implementation at the time of writing; Appendix B records the same status, since the papers in this series are live simultaneously and describe progress at different moments.)

Threats to validity

It is worth naming the objections directly rather than leaving them to be inferred, and saying for each one what would answer it.

Designer bias. The architecture was designed, implemented, and evaluated by the same person. The judgments that served as ground truth — which artifacts were relevant, which transformation was better grounded — are that person’s. Nothing here rules out the possibility that the model works because its author knows what it expects. Only a transformation carried out by someone who did not design the architecture can settle that.

Single domain. The transformations reported were run against one reference domain. A domain chosen by the architect may be one the architecture happens to fit. The claim that progressive enrichment terminates in admissible artifacts for *any* domain is an architectural argument (§9, §16), not an empirical result, and should be read as such until a domain the author did not choose has been transformed.

No comparative baseline. No matching change was carried out through a traditional specification-driven process for comparison, so no claim is made about speed, cost, defect rate, or effort. The claims here are structural — that certain classes of drift cannot occur — not quantitative.

Construct validity of “no specification.” A skeptic may reasonably say the specification has been renamed rather than removed: the enrichment registers *are* a specification, written in stages. The answer is that the distinguishing property was never the absence of specifying content but the absence of a *separately authored, separately maintained* artifact — the derived registers compile, are validated mechanically, and cannot be edited into disagreement with the system without failing admission. Whether that distinction holds up under adversarial use is an open question.

Worker substitutability. Worker independence was exercised across an automated model worker, an interactive session, and deterministic replay — three workers, one author, one instruction set. That the guarantees survive a worker chosen adversarially, or one substantially weaker, is argued from the Knowledge Partition rather than demonstrated.

Maturity. The reference implementation is young, and the reported transformations are few. An architecture that governs change well over dozens of transformations may still fail to over thousands; scale effects on the transformation record, on promotion latency, and on the cost of re-deriving a large baseline are unmeasured.

16. Architectural properties

The architecture can be summarized as a set of properties, each one a consequence of a section above.

- **Baseline-first.** There is always a baseline; it defines the system completely.
- **Transformation, never greenfield.** Every executable state is reached by transforming an existing baseline.
- **One producer of meaning.** Only the human-directed author creates meaning; the platform never does.
- **Provenance everywhere.** Every derived fact declares its source; none is merely asserted.
- **Progressive enrichment.** Each stage adds one layer of meaning and contradicts no earlier stage.
- **Authority without authorship.** The human owns every decision and authors no specification.
- **Promotion as closure.** The loop closes only when a validated candidate is promoted.
- **No separately authored specification.** Specifications remain, as governed declarations; the separate authored one is gone.
- **Implementation independence.** The model names no language, runtime, or platform.
- **Worker independence.** Human, AI, and hybrid workers are interchangeable.
- **Knowledge accumulation.** Each transformation leaves the system knowing more than before.

Read one way, these are properties this architecture happens to have. Read the other way — the more useful one for anyone who wants governed transformation without this architecture — they are **requirements**. An implementation conforms to the architecture, whatever its technology, if and only if it provides all of the following:

- **an authoritative executable baseline** — a single compiled artifact that defines the system completely and is the only thing the runtime executes;
- **a single origin-agnostic admission arbiter** — one compiler that decides buildability over Baseline U Candidate Delta and cannot tell new from old;
- **behavioral validation against a declared outcome** — a governed stage, distinct from admission, that checks the realized candidate's execution against the acceptance boundary the problem declared;
- **provenance on every derived fact** — no property asserted without a source, structural or decided;
- **a meaning partition** — deterministic knowledge produced only by the platform, judgment produced only by the author;

- **promotion as the sole state change** — the system changes only when a validated candidate is installed as the new baseline, and installation judges nothing.

Everything else in this paper is a consequence of these six. A realization that supplies them is a governed-transformation architecture; one that omits any of them is not, whatever it is called. This is the exact sense in which the architecture is separable from its reference implementation: the requirements say what must be true, not how the reference implementation makes it true.

17. Architectural consequences

The properties of §16 describe what the architecture *is*. It is worth stating separately what *follows* from it — the consequences for software engineering if the architecture is correct. These are not offered as marketing. Each is a direct consequence of a requirement already established, and each is meant to be checkable against the argument rather than admired.

If a system is built this way, then:

- **The specification ceases to be a first-class engineering artifact.** What was the pivot of every prior methodology becomes a derived projection with no independent existence (§3, §12).
- **Software engineering becomes baseline transformation.** The unit of work is no longer a document or a diff but the governed transformation of one executable baseline into the next (§5, §6).
- **Governance becomes executable rather than advisory.** Boundaries, ownership, and invariants are compiled facts the system enforces, not opinions a board records after the fact (§4, §13).
- **Architecture accumulates knowledge instead of decaying.** Because every derived fact carries provenance, each transformation leaves the next one more informed, reversing the direction in which documentation normally rots (§8, §14).
- **Workers become interchangeable.** Human, AI, and hybrid sources of judgment present the same governed interface, so no guarantee depends on who or what supplies the judgment (§7, §11).
- **Software evolution becomes closed-loop.** What comes out is the same kind of artifact as what went in, and it is checked against what went in, so evolution composes cycle after cycle instead of drifting (§10).

Each of these is a loss of something the field long treated as permanent — the authored specification, the review-board veto, the decaying document — and a gain of something it long treated as unattainable: a description that cannot drift because it is never separately held.

18. Why the transformation endures

There is a deeper reason the architecture is built to last, and it is worth drawing out on its own, because it is the payoff of the abstract's central claim — that the transformation, not any document and not the code, is the thing that endures.

Consider what a traditional system actually is: a tower of mortal artifacts. Requirements sit above a specification, which sits above code, which sits above a framework, which sits above a language and a runtime. Every layer in that tower has a finite life. Languages fall out of fashion; frameworks are abandoned; code is rewritten; even the specification — supposedly the durable one — is the first to rot, because its agreement with the system was only ever maintained by diligence, and diligence stops the moment attention moves on. Worse, the layers are coupled: when a lower layer dies, the layers above it are stranded, describing a system that no longer runs the way they claim.

Governed transformation inverts the tower. The enduring artifact is not any layer of the stack but the transformation that produces the stack. The snapshot, the implementing code, the runtime, the very language they are written in — each is a *product* of a transformation, and a product can be regenerated. If the runtime is replaced, the same transformations reproduce the system on the new one. If the implementation language changes, the contracts are unchanged and re-realized beneath them. What must persist is only the chain of governed transformations and the provenance they carry, because from that chain every lower layer is re-derivable, and without it no lower layer can be trusted.

This is the precise sense in which an architecture, unlike an implementation, does not age. The Baseline Transformation Principle already said that the system *is* its baseline; this section adds that the baseline is disposable in a way the transformation is not. Baselines are meant to be superseded — that is what promotion does. The transformation that reaches them is the one thing the architecture is designed never to throw away. A reader twenty years from now, on hardware and in languages not yet invented, could take the recorded transformations and rebuild the system faithfully, because everything needed to do so was derived, not remembered.

19. Implications

For software engineering, the synchronization problem simply ends, because the artifact that created it no longer exists. For enterprise and systems architecture, governance boundaries become computable, versioned facts rather than the opinions of a review board. For digital transformation, the phrase acquires a precise meaning: the governed transformation of an executable baseline into the next one. And for AI-assisted engineering, the division of labor is settled by construction rather than by policy — judgment to the worker, deterministic knowledge to the platform, authority to the human — so that even a highly capable automated worker can be trusted without ceding architectural authority to it.

These implications are offered at the strength the work supports. The architecture has been exercised thoroughly in one reference domain. That is enough to demonstrate the model and to make it worth adopting and testing more widely. It is not yet a claim about every domain, and the paper makes none.

20. Future work

The most natural next step turns the transformation process into a governed workflow *inside* the system itself, so that submitting a business problem becomes an ordinary governed act — at which point the loop is closed not only in concept but in mechanism, and the system governs its own evolution with the same machinery it uses for everything else.

Further out lie a family of governed views over the system's own model, computed rather than hand-built; workers that carry more of the transformation autonomously; and governance distributed across federated boundaries. None of these is required for the model presented here. Each is, fittingly, a transformation of it — to be carried out through the same loop this paper describes.

21. Conclusion

The earlier work in this series established that behavior belongs in protocol rather than in implementation, and then that evolution could be governed rather than left to habit. This paper draws those results to their natural end.

Construction, execution, and evolution stop being separate disciplines. They become three governed compilations over one architecture. Once software is seen as a succession of governed transformations rather than a succession of implementations, the software specification stops being the enduring engineering artifact. The governed transformation becomes that artifact.

What this architecture requires is stated here; what it takes to realize it, and what realization reveals that architectural review does not, is the subject of the third paper in the set [Ganti, 2026m].

Appendix A — Key Terms

Business Problem. What the human supplies to a transformation, together with the authority to resolve the decisions it raises. Business intent, governance intent, and design intent are all derived from it; the human authors no specification.

Baseline Transformation Principle. Every executable state of a protocol-governed system is reachable only through a governed transformation of an existing executable baseline. Corollary: if the baseline does not change, the system is unchanged. Stated as a principle, not a theorem: it has no formal apparatus, and the prior work states the same content as a definition.

Knowledge Partition. The rule that only the human-directed transformation author may create meaning. The platform may compute deterministic knowledge but may never assign semantic relevance.

Derivation Provenance. The discipline that every derived fact declares its source — a structural fact or a governed decision — and is never merely asserted.

Progressive Semantic Enrichment. The mechanism by which each stage of the transformation adds one layer of meaning and contradicts no earlier stage, raising an informal problem toward an executable protocol.

Candidate Artifact. A proposed piece of protocol that the transformation produces but that does not yet officially exist, pending admission.

Admission. The act by which the admission compiler — the same constitutional compiler that governs everything else, operating over Baseline U Candidate Delta — determines that candidate artifacts are buildable. The sole arbiter of *admissibility* (structural and constitutional soundness); it does not judge behavior.

Execution Validation. The governed stage, distinct from admission, that runs the admitted and realized candidate and checks its observed execution against the outcome the business problem declared. The sole arbiter of *behavioral adequacy*. The loop closes semantically only when validation passes.

Outcome. The acceptance boundary a business problem declares at the outset, against which execution validation later judges whether the candidate resolves the problem.

Promotion. The installation of an admitted and validated candidate as the new baseline; the moment the system changes. It verifies integrity and re-judges nothing — where realized by a snapshot rebuild, the rebuild introduces no new judgment.

Protocol Snapshot. The compiled, authoritative artifact that defines what the system currently is; the baseline every transformation transforms.

Authority without Authorship. The division of labor in which the human owns every decision requiring authority and authors none of the specification that expresses it.

(Terms carried from the prior papers — Change Request, dossier, purity ladder, Business Model, Business Intent, Governance Intent, Design Intent, Authoring Mandate — retain their published definitions. “Governance Dividend” is omitted: it is defined differently across the prior papers, and §14 describes the phenomenon without it.)

Appendix B — The Shape of a Governed Transformation

The body argues why the architecture takes this form. This appendix states the form itself, without argument and without reference to any implementation's names, so that a reader can recognise a conformant realization or build one. The conceptual model is what endures; repository layouts, command surfaces and artifact formats do not, and none appear here.

Why a sequence at all

A business problem is informal and a compiled baseline is exact. Nothing bridges that distance in one move, and the attempt is what the body diagnoses: someone writes the artifacts directly, and the reasoning that produced them exists only in their head.

Staging replaces that with a **structured enrichment of the problem statement toward the baseline it must align with**. Each step adds one layer of meaning and is checked against the step before it, so the author's burden shifts from *authoring artifacts* to *answering questions* — and the artifacts become a consequence of the answers rather than a separate act of creation.

Two properties follow, and they are the reason to accept the ceremony of stages:

- **A fault is localised.** When a derivation is wrong, the stage that first stated the wrong thing is identifiable, because every stage's output was checked against its input. A single-pass derivation offers nowhere to look.
- **Reproducibility is observable.** Because each step is judged by a declared rule set rather than by taste, the same answers against the same baseline yield the same artifacts, and any divergence is attributable to a step rather than to a person having a different day.

A reader may reasonably ask how many stages. The answer is not a constant; it is however many are needed for each to add exactly one kind of meaning and no more. The reference implementation uses nine, which is offered below to ground the shape — not as a required number.

The stages, as the reference implementation names them

Each stage establishes one thing and is judged before the next may begin. The groupings map onto the altitudes of §9.

Business Problem

- | | | |
|----|----------------|-------------------------------------|
| P0 | Seed | what the business said, reorganised |
| P1 | Change Request | the seed restated, each row cited |

Business Understanding

- | | | |
|----|---------------|---|
| P2 | Domain Model | what already exists, verified against the baseline rather than asserted |
| P3 | Analysis Loop | the decisions: reuse it, extend it, or |

P4 Business Model author it new
 those decisions consolidated

Business Intent

P5 Business Intent what the subdomain will be

Governance Intent

P6 Governance Intent which part of the system owns what

Design Intent

P7 Design Intent the artifacts, their identities, wiring

Construction

P8 Authoring Mandate what to build, and in what order

The lower stages speak only business language; the upper ones may name identities. That ordering is the purity ladder of §9, and it is why a stage that reaches for vocabulary above its rung is detectable rather than merely discouraged.

Two lifecycles, and the boundary between them

The pipeline is where a change is carried to term; construction is where it takes material form — and nothing is built until the design that determines it is complete.

That boundary is the whole reason the two are drawn separately: everything to the left produces *meaning about* a change and nothing that runs, and everything to the right produces artifacts and originates no meaning.

Design Pipeline (P0–P8)

Input:

Problem statement

Process:

P0 Change Seed
 P1 Change Request
 P2 Domain Model
 P3 Analysis Loop
 P4 Business Model
 P5 Business Intent
 P6 Governance Intent
 P7 Design Intent
 P8 Authoring Mandate

Output:

A design that determines its artifacts, its verdicts, and the mandate to build them

Construction

Input:

P7 Design Intent
 P8 Authoring Mandate

Process:

Determination measured
 – refuse below threshold
 Materialization (render)
 Promotion (a separate, deliberate act)
 Compiler validation
 Assembly
 Execution validation

Output:

Protocol artifacts, a compiled baseline contribution, and behaviour observed against the declared outcome

Four things in the right-hand column are worth reading closely, because each is a place a conventional build would have proceeded and this one does not.

Determination is measured first, and construction refuses below the threshold. This is the gate that keeps the generator from becoming a second author (§9).

Materialization renders the artifact from the design alone — never from the design merged with whatever is currently installed. That is what makes the same design build identically against two systems.

Promotion is a separate, deliberate act. Rendering writes to a staging area; installing is its own step, and the gap between them is where a human decides. Collapsing them would make building and changing the system the same event, which §10 argues they must never be.

Execution validation closes it. Artifacts that compile and assemble prove only that they are well-formed. The lifecycle is not complete until the system has run against real state and the outcome the problem declared has been observed.

The sequence

A transformation is an **ordered sequence of gated steps**. Each step:

- produces exactly **one document**, composed of *registers* — named tables with declared columns. Prose in such a document is commentary and carries no governed content.
- is judged by a **rule set declared as data**: each rule names itself, the register it governs, the mechanism that evaluates it, and that mechanism's parameters. The set of mechanisms is closed, and an unrecognised one fails rather than being skipped.
- yields a **binary verdict**. Every finding names the rule and the location. A count is not a verdict, and no rule is skipped once another has failed.
- **hands off explicitly**: it declares what it emits and what its successor consumes. A handoff that cannot be checked is reported as unchecked rather than passing silently, because an unchecked handoff and a preserved one are otherwise indistinguishable.

Steps differ in kind. Some decide; some only restate what precedes them. A step whose content its predecessor uniquely determines is produced mechanically rather than authored, and refuses to run against a predecessor that was not admitted.

What is checked between steps, not within one

- **Preservation, in both directions.** A step may not drop what its predecessor committed to, and may not state what its predecessor does not. Checking only the first admits invention; checking only the second admits loss.

- **Grounding.** A claim about what the existing system already provides is read from a named, frozen baseline rather than asserted. Business truth, belief about what exists, and open question are kept in separate registers: a belief recorded as a truth is never verified, and a question recorded as a truth is answered by invention.
- **Identity discipline.** A register may not name an implementation identity until the step that assigns identities. Where a capability is named before it is identified, the provisional name and the identity later bound to it are reconciled in both directions.

Where humans are engaged

- A question the transformation cannot answer is **recorded as a question against a named owner**. It is never filled in, and never hedged with a placeholder — a cell stating that a question is unanswered reads as decided to every later step.
- An unresolved question that blocks progress makes its document **inadmissible**. Otherwise the next step answers it by invention.
- Human semantic content enters **once**. A later step preserves it, references it, or declares that it has superseded it. Silent replacement is refused.
- **Acceptance is not admissibility.** A gate is a person accepting a document; a verdict is a rule set admitting one. A document may be admissible and unaccepted, and the two are never conflated.

From design to artifacts

- **Determination is measured before anything is built** — the proportion of facts an artifact requires that the design actually states — and construction refuses below the declared threshold. A generator that supplies a fact the design omits has become a second, ungoverned design authority.
- A built artifact is a **function of the design alone**, never of the design and the current system. Otherwise the same design builds differently against two systems and determination means nothing.
- It follows that **amending an existing artifact is a whole redeclaration**, not a patch — and that a redeclaration is compared against what it replaces and refused if it narrows it.
- **Scheduling and amendment are different acts**, and both are realized. An artifact that already exists cannot be scheduled for authoring, so a loop that realizes only what it schedules silently leaves amendments unapplied — which is every change after the first one.
- Everything the transformation asked for is built, and everything built traces to something it asked for. Both directions are checked.

What proves it

Document admissibility, determination in full, and faithful construction are **jointly insufficient** to show that anything works. A transformation is complete only when the system it produced has been executed against real state and the stated acceptance

criteria observed. An acceptance criterion asserts the state it claims, never the status a call returned; a criterion satisfied by a success code over an operation that did the wrong thing is not satisfied, and a rule that passes because a value was absent has not passed.

What was exercised, and the limits of it

The architecture was realized and exercised as a family of governed transformations in a reference domain, carried from informal business problems through construction, admission, promotion and execution against real state. A later transformation *extended* what an earlier one had built — amending existing artifacts rather than authoring new ones — which is the case every change after the first one presents and the one that exercises preservation, grounding and amendment together.

Two observations bear on the paper’s claims, stated with their limits (see §15). When a structural pre-filter was used to guess which parts of a change’s neighborhood were relevant — a judgment the Knowledge Partition reserves for the worker — it surfaced artifacts the author later discarded and omitted ones the author kept. This is an illustration consistent with the partition, not a proof that relevance cannot be computed from structure; the in-principle argument in §7 carries that weight. And the evidentiary depth of a transformation was raised toward parity with a hand-authored reference by relocating evidence acquisition from the worker into a governed platform capability, without changing the worker, its instructions, or the validation oracle — the platform, not the worker, improved.

Both observations come from a single reference domain and a single practitioner who is also the architecture’s designer. They are existence proofs, not a controlled evaluation.

Worker independence was exercised across an automated model worker, an interactive human-and-assistant session, and deterministic replay of a recorded response. The governed artifacts produced were identical in authority regardless of the worker.

The reference implementation is open source at <https://github.com/protocol-governed-computing>: the design compiler that carries a business problem through its stages, the construction compiler that turns a validated design into artifacts, the protocol compiler and snapshot assembler that admit them, and the governed domains the transformations in this paper were run against. Each repository carries an architecture document written for a reader approaching it without prior familiarity, and a reader may re-run a transformation rather than take these paragraphs’ word for it.

Appendix C — References

Companion papers

Ganti, B. (2026l). *Protocol-Governed Computing: An Architecture for Deterministic Declarative Execution*. Companion paper. DOI: <https://doi.org/10.5281/zenodo.21879516>

Ganti, B. (2026m). *Protocol-Governed Computing: Realizing the Normative Platform and Its Governed Transformation*. Companion paper.

Prior work

Ganti, B. (2026). *Protocol-Governed Systems: Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20300611>

Ganti, B. (2026). *Protocol-Governed Systems: Compiler Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20471804>

Ganti, B. (2026). *Protocol-Governed Systems: Runtime Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20478471>

Ganti, B. (2026). *Protocol-Governed Systems: Architecture Inversion Concepts*. DOI: <https://doi.org/10.5281/zenodo.20497732>

Ganti, B. (2026). *Protocol-Governed Systems: Closed-Loop Governed Evolution (v1)*. DOI: <https://doi.org/10.5281/zenodo.21434335>

Beck, K., et al. (2001). *Manifesto for Agile Software Development*. agilemanifesto.org.

North, D. (2006). *Introducing BDD*. Better Software Magazine. (Representative of the executable- specification / behavior-driven lineage discussed in §12.)

Brooks, F. P. (1987). *No Silver Bullet: Essence and Accidents of Software Engineering*. IEEE Computer, 20(4), 10–19.

Schmidt, D. C. (2006). *Model-Driven Engineering*. IEEE Computer, 39(2), 25–31.

Mernik, M., Heering, J., & Sloane, A. M. (2005). *When and How to Develop Domain-Specific Languages*. ACM Computing Surveys, 37(4), 316–344.

Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.

ISO/IEC 20000-1:2018. *Information technology — Service management — Part 1: Service management system requirements*.

AXELOS. (2019). *ITIL Foundation: ITIL 4 Edition*. TSO.

Lamport, L. (1994). *The Temporal Logic of Actions*. ACM Transactions on Programming Languages and Systems, 16(3), 872–923.

Appendix D — Worked Example: Transforming a Chain Subdomain

This appendix walks one real transformation from beginning to end, so that every stage the body described in the abstract can be seen concretely. It is the transformation that authored a canonical **chain** subdomain — a serial, hash-linked, immutable ledger of finalized blocks — into a blockchain reference domain that already contained consensus, block, mempool, and related subdomains. The change-management engine that drives this transformation is open source at <https://github.com/protocol-governed-computing>.

Throughout, watch for the one thing the architecture insists on: at no point does a human author a specification of the chain. The human states a problem and makes decisions; everything else is derived.

The baseline. Before the transformation, the domain already had a compiled, authoritative snapshot: a Proof-of-Stake consensus mechanism, a shared block entity, a mempool of pending transactions, and an orchestration subdomain. This is the baseline the transformation begins from. Nothing here is greenfield.

The business problem. The human supplies a problem, in business language: the domain needs a permanent, tamper-evident record of the blocks that consensus has finalized — a chain — so that the history of the ledger is serial, immutable, and independently verifiable. The human also states what already exists that the chain must respect (finalized blocks come from consensus; a genesis starting point is required) and what is out of scope (the chain does not decide *which* blocks finalize; it only records them). No design accompanies this. It is a statement of need and boundary. It also carries a declared **outcome** — the acceptance boundary against which the finished transformation will later be judged: a finalized block can be appended, the chain can be read back in order, genesis is established exactly once, and a block that does not link to the current head is rejected. That outcome is what execution validation will check before anything is promoted.

Business understanding (what exists). The platform deterministically gathers the neighborhood of the problem — the existing block entity, the consensus outputs, the events already defined, the storage already in use. It presents this evidence. It does *not* mark any of it “relevant”; that judgment is the worker’s. The author, holding the problem, disposes of each item: this block entity is relevant, this consensus event is relevant, this unrelated subdomain is not.

Business intent (what may happen). From the disposed evidence and the problem, the transformation derives behavior: a finalized block may be appended to the chain; the chain may be read; genesis initializes the chain exactly once; an append must reject a block that does not link to the current head. These are stated as governed behavioral rules, in business vocabulary, with no identifiers or storage yet. Note that the human did not write these rules as a specification — they were derived from the problem, and the human’s authority enters only where a genuine decision is required, for instance that a broken link is *rejected* rather than *reordered*. That decision is real semantic content, and the human makes it; what the architecture changes is not that the human decides less, but that the

decision is captured as a governed rule with its provenance and projected into the artifacts, so nothing downstream can quietly disagree with it.

Governance intent (who owns what). The transformation now places the behavior. The chain is its own subdomain, a peer of consensus rather than a part of it, because the chain records the output of consensus but does not participate in it. Ownership boundaries are drawn: the chain owns the ledger record; it reads finalized blocks but does not write them. Where the placement raises a decision that needs authority — is chain a peer or nested? — the human decides; the rest is derived from the behavior established upstream.

Design intent (concrete shape). Only now do concrete structures appear: the identifiers for the new capabilities, the shape of the chain record, the events the chain emits, the storage it owns. Every one of these is derived from the intent above it, and every one carries its provenance back to a rule or a decision. Nothing is invented that does not trace to something earlier.

Construction, admission, and implementation. The design is compiled by the semantic compiler into *candidate* artifacts — proposed capabilities, events, and records that do not yet officially exist. The admission compiler — the same constitutional compiler that governs the rest of the domain — assembles the whole domain with the candidates overlaid and asks whether it can consume them. It cannot tell which artifacts are new. It either admits the whole or reports precisely what does not fit. Here, admission required resolving a type that the design had left implicit; the transformation returned to design, resolved it, and re-admitted. This return is not the pipeline leaking — it is the “no stage contradicts an earlier one” rule working exactly as designed: the gap was fixed at its source rather than patched at the end. With the candidates admitted, the declared capabilities are realized in conventional code that satisfies their contracts — the append, the read, the genesis initialization — with conformance checked by the compiler, so the implementation cannot drift from the contract it fills. The realized candidate is then run against the outcome the problem declared: a block appends, the chain reads back in order, genesis initializes once, a non-linking block is rejected.

Promotion (prospective). Promotion is the step that would install the admitted and validated candidates as the new baseline: it copies the admitted artifacts into place and changes nothing about their correctness, which was settled at admission and validation. At that point the snapshot would contain a chain subdomain, the domain would have a new baseline, and the system would have changed. In this walkthrough the transformation was carried through construction and admission; promotion is described here from the six earlier transformations that were carried through it in full. Until promotion happens, the validated chain remains a candidate, and the domain is — by the Baseline Transformation Principle — unchanged.

The new baseline, and the next problem. Once promoted, the snapshot becomes the baseline from which the next transformation begins. And the domain would know more than it did: the boundary decision that made chain a peer, the rule that appends must link

to the head, the record of why each artifact exists — all of it becomes governed structure that the next transformation inherits rather than rediscovers.

At no point in this walkthrough did a human write a specification of the chain. The human stated a problem with its acceptance boundary, made a handful of decisions that required authority, and reviewed and approved the derivation at the governed gates. Everything that a specification would traditionally contain — the behavior, the boundaries, the structure, the artifacts — was derived, under governance, with its provenance intact. That is the architecture, in one concrete case.